Lab #2

Design with High Level Synthesis on Arty Z7 SoC Development Platform

Oct. 12, 2026

Vitis HLS and Vitis IDE

Feature	Vitis IDE (Used in Lab #1)	Vitis HLS (for Lab #2)
Purpose	**Software Development** Environment	**Hardware Design** with High-Level Synthesis
Target	The **PS - Processing System**	The **PL - Programmable Logic**
Input	Standard C/C++ application code (with a `main()` function)	Synthesizable C/C++ function(s), guided by `#pragmas`
Main Goal	To **control** existing hardware and run software logic.	To **create new**, custom hardware accelerators.

SystemVerilog

Verilog	System Verilog
wire	wire / logic
reg	logic
always @(*)	always_comb
always @(posedge clock)	always_ff @(posedge clock)

```
module 1b_register(
    input rst,
    input clk,
    input in,
    output out
);
    always_ff @(posedge clk, posedge rst) begin
        if (rst) begin
            out <= 1'b0;
        end
        else begin
            out <= clk;
        end
    end
endmodule
```

Figure: 1b register in SystemVerilog

```
always_comb begin
    case (var)
        3'b0: begin
            // do sth
        end
        3'b101: begin
            // do sth
        end
        3'b111: begin
            // do sth
        end
        default: begin
            // do sth
        end
    endcase
end
```

Figure: "case" example in SystemVerilog

SystemVerilog

- Use a generate block to build the hardware

```
1  generate
2      genvar i;
3      for (i=0; i<N; i++) begin
4          one_bit_adder (
5              .a(a[i]), .b(b[i]),
6              .cin( carries [i]),
7              .sum(sum[i]),
8              .cout( carries [i+1])
9          );
10     end
11 endgenerate
```

- The tool will "elaborate" the design
- It will evaluate "if" statement and unroll "for" loops

Pipelined Multiplication

Partial Products

- Multiply the first *n* bits of the two components
- Multiply the next *n* bits, etc.
- Sum the partial products to get the answer

Example: 4-stage Pipelined Multiplication

```
    multiplicand: 00001011   << 2
      multiplier: 00000111   >> 2
partial product: 00100001
    multiplicand: 00101100   << 2
      multiplier: 00000001   >> 2
partial product: 00101100
                + 00100001
                = 01001101
```

Stages

- C Simulation
  - Goal: Fast functional verification and validate the logic of source code using testbench.
  - Testbench: success with `main()` function return 0 and fail with `main()` function return nonzero.

```
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] *************** CSIM finish ***************
Finished C simulation.


@E Simulation failed: Function 'main' returns nonzero value '1'.
ERROR: [SIM 211-100] 'csim_design' failed: nonzero return value.
INFO: [SIM 211-3] *************** CSIM finish ***************
```

Stages

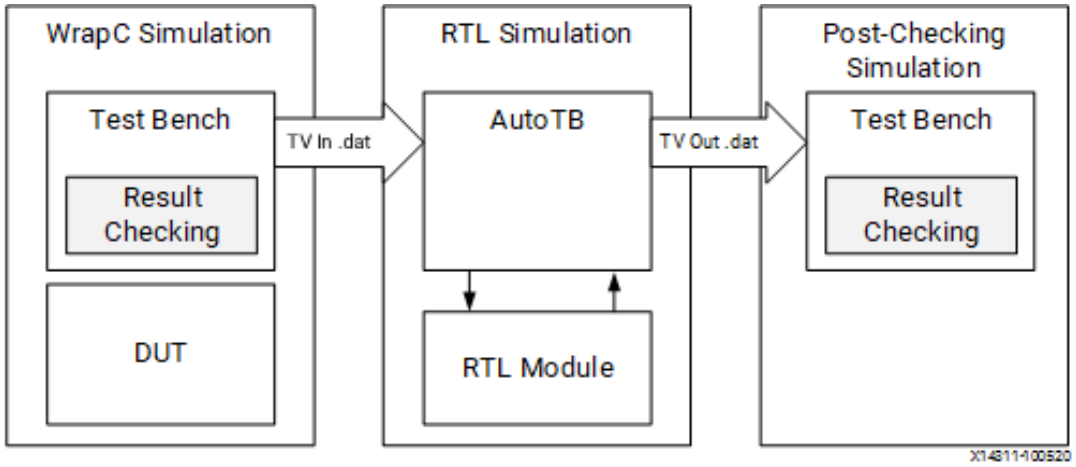
- C Synthesis
  - **Translate** C to RTL (Verilog/VHDL) and **synthesize** it for the targeted chip and the targeted frequency, and generates reports.
  - Reports:
    - Synthesis: reports the synthesis results with information on quality of results, HW interfaces, burst transactions and more.
    - Function Call Graph: Displays the HLS component after C Synthesis or C/RTL Co-simulation to show the throughput of the design in terms of latency and II.
    - Schedule Viewer: Shows each operation and control step of the function, and the clock cycle that it executes in.
    - Dataflow Viewer: Only available when the `DATAFLOW` pragma or directive is used in the design.
    - Kernel Guidance: reports Guidance messages to provide design advice for the source code and synthesis results.

Important notice

- Do not directly use the RTL files generated in the `syn/Verilog` or `syn/vhdl` folder for synthesis in the Vivado tool. Instead you must use the exported output files generated by the package process as described in Packaging the RTL Design. The HLS compiler sometimes uses IP from the Vivado IP catalog in the synthesized RTL code, such as with floating point designs, and the `verilog` and `vhdl` folders will not contain the complete design.

Stages

- C/RTL Co-Simulation
  - Goal: Verify the generated RTL against the original C++ testbench.



Source: [Vitis HLS Synthesis User guide]

1. Generate **inputs**: “input vectors” to DUT(top-level function).
2. Simulate **hardware**: feed those “input vectors” to RTL created by *C synthesis* and store corresponding “output vectors”.
3. Verify **outputs**: return those “output vectors” to `main()` function of C testbench to verify the results.

Lab #2

Goals

- Get familiar with the concept of High-level Synthesis (HLS)
- Learn to use HLS to create an IP block in C/C++
- Learn about differences between programming with HLS vs. Verilog in terms of design quality
- Get started with the Arty Z-7 board as a Python accelerator and be able to run example Jupyter notebooks with the board
- Load PYNQ overlays, use overlays and create overlays

Deliverables

- Details are in the Lab #2 description
- Peer evaluation form (individual grade)

References

- Yichen Cai, 2023 Lab#2
- Fan Hu, 2024 Lab#2
- https://docs.amd.com/r/en-US/ug1399-vitis-hls/Introduction